Conductor: Linux Containers from Scratch

A Systems Implementation of Namespaces, Filesystem Isolation, Image Layering, and Virtual Networking

Pratik Wadekar
Department of Computer Science & Engineering
Systems Engineering Technical Report

Abstract

Containers are widely used to package and isolate software applications, but they are often misunderstood as virtual machines. Unlike virtual machines, which run an independent operating system kernel on virtualized hardware, Linux containers are standard host processes isolated through kernel mechanisms. This report details the architecture and implementation of **Conductor**, a container toolkit built from first principles. Developed across four progressive milestones, the project covers: (1) low-level C namespace orchestration via `clone`, `setns`, and `pidfd_open`; (2) filesystem sandboxing via `chroot` and CPU resource limits using control groups (cgroups v2); (3) a container management engine supporting image compilation, content-addressed layer caching, and OverlayFS union mounts; and (4) a software-defined container networking stack with virtual Ethernet pairs, network address translation (NAT), port exposure, and multi-container microservice orchestration. The report explains the underlying kernel interfaces, design trade-offs, and practical debugging challenges encountered during system implementation.

1. Introduction and Systems Motivation

Modern deployment workflows rely extensively on containerization to run isolated, repeatable services. While container runtimes present a unified command-line experience, their core functionality is achieved by composing multiple independent Linux kernel subsystems.

Treating container runtimes as black boxes obscures how processes are isolated, how images are layered, and how virtual network traffic is routed. The primary goal of the Conductor project is to build containerization mechanisms from scratch to understand how these subsystems interact:

- **Namespaces**: Partition global kernel resources (process IDs, mount trees, network interfaces, hostnames, and IPC handles) so each container operates in an isolated context.
- **Control Groups (cgroups v2)**: Monitor and restrict resource consumption, such as CPU bandwidth and memory allocation.
- **Union Filesystems (OverlayFS)**: Enable efficient, copy-on-write image layering and fast container instantiation without copying entire root filesystems.
- **Virtual Networking (`veth` and Netfilter)**: Connect isolated network namespaces to the host network stack and enable communication via routing and packet translation.

2. Progressive Architecture and Implementation Milestones

Conductor was constructed through four progressive stages, moving from raw system calls to a multi-container microservice environment.

2.1 Task 1: Syscall-Level Namespace Mechanics

The initial milestone focuses on understanding how Linux namespaces are created, inherited, and joined at the C system-call level (`task1/namespace_prog.c`).

- **Namespace Creation via `clone(2)`**: Child processes are spawned with explicit namespace flags (`CLONE_NEWUTS` and `CLONE_NEWPID`) using a dedicated, downward-growing stack space allocated via `malloc`.
- **Asynchronous Namespace Attachment via `setns(2)`**: The implementation demonstrates a critical property of PID namespaces: attaching a parent process to an existing PID namespace does not change the parent's own PID; rather, only subsequent children created by `fork(2)` become members of the target PID namespace.
- **Synchronization and Process Descriptors**: To avoid PID reuse races inherent in path-based references under `/proc/<pid>/ns/pid`, the implementation uses Linux `pidfd_open(2)` descriptors, synchronizing execution steps across processes using unidirectional UNIX pipes.

2.2 Task 2: Filesystem Sandboxing and Cgroups Resource Limits

Task 2 advances from process view isolation to filesystem containment and CPU bandwidth limiting via the script `simple_container.sh`:

- **Filesystem Confinement with `chroot`**: Isolates the root directory for a process. To allow dynamically linked C binaries to execute inside the root without copying the entire operating system, dynamic library dependencies identified via `ldd` are resolved and bind-mounted into the root.
- **Cgroups v2 Bandwidth Throttling**: A control group is created under `/sys/fs/cgroup/simple-container` with the `cpu` controller enabled. The Completely Fair Scheduler (CFS) bandwidth parameters are configured with a 50% quota:

$$\text{CPU Limit} = \frac{\text{cpu.max (quota)}}{\text{period}} = \frac{50\,000\,\mu\text{s}}{100\,000\,\mu\text{s}} = 50\% \text{ of 1 CPU Core} \tag{1}$$

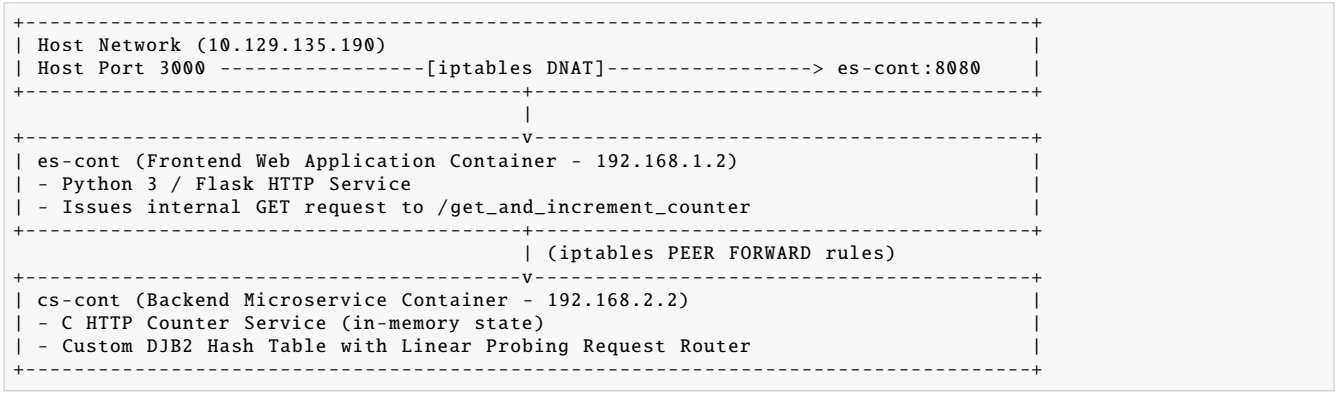

Figure 1: Task 4: Two-Tier Containerized Microservice Architecture and Network Routing Flow

The executing shell PID is registered in `cgroup.procs` prior to executing `unshare`, ensuring that all subsequent container child processes automatically inherit the CPU cap.

2.3 Task 3: The Conductor Container Engine

Task 3 unifies these isolation mechanisms into a command-line container engine (`task3/conductor.sh`) supporting full container lifecycles:

- **Base Image Bootstrapping**: Automates `debootstrap` initialization of Debian Bookworm or Ubuntu distributions into cached base layers.
- **Conductorfile Parser**: Implements a build recipe parser supporting `FROM`, `RUN`, and `COPY` instructions.
- **Ephemeral Namespace Build Sandbox**: Each `RUN` build step runs inside a transient namespace over a temporary OverlayFS mount, capturing created and modified files into distinct layer directories.
- **Container Runtime Execution**: Launches containers using:

```
unshare --uts --pid --net --mount --ipc \
        --mount-proc --fork --kill-child -R <merged_dir>
```

  The `--mount-proc` and `--fork` flags ensure that the child process becomes PID 1 inside its own PID namespace with a private `/proc` mount, while `--kill-child` guarantees that container subprocesses terminate when the main process exits.
- **Container Inspection and Execution**: Supports inspecting running containers (`conductor.sh ps`) and entering active namespaces (`conductor.sh exec`) using `nsenter -a -r -w`.

2.4 Task 4: Software-Defined Networking and Microservice Deployment

Task 4 demonstrates multi-container networking and application orchestration using Conductor's built-in networking subsystem:

- **Virtual Ethernet Pairs**: Allocates `veth` interface pairs connecting the host root network namespace to the container's network namespace.
- **Subnet Allocation**: Assigns dedicated /24 IPv4 subnets from the private 192.168.0.0/16 address space.
- **NAT and Port Forwarding**: Configures outbound internet routing using iptables `MASQUERADE` and inbound traffic exposure using dual-rule destination NAT (DNAT).
- **Two-Container Deployment**: Deploys a C-based HTTP counter microservice in one container and a Python Flask frontend in a second container, connecting them over a peered virtual network.

3. Core Engineering Mechanics

3.1 OverlayFS Union Mount Architecture

Rather than duplicating entire root filesystems on each container launch, Conductor employs the Linux `overlayfs` union filesystem driver. An image consists of an ordered chain of immutable read-only directories:

$$\text{Merged Rootfs} = \text{Upper (rw)} \cup \bigcup_{i=1}^{N} \text{Lower}_i \cup \text{Base (ro)} \tag{2}$$

- **Lower Directories (`lowerdir`)**: Read-only layer stack containing the base OS and successive build diffs.
- **Upper Directory (`upperdir`)**: A private writable directory per container that captures all file creations, modifications, and deletions.
- **Work Directory (`workdir`)**: A staging directory on the same underlying filesystem used by the kernel to prepare atomic file transitions.

3.2 Content-Addressed Build Layer Caching

To prevent repeating package installations and file copying during iterative builds, Conductor uses a content-addressed caching mechanism. Each layer hash H_{layer} is derived from its parent layer hash and instruction content:

$$H_{\text{RUN}} = \text{SHA256}\Big(\texttt{"RUN-"} \parallel H_{\text{parent}} \parallel \text{SHA256}(\text{Command})\Big) \tag{3}$$

$$H_{\text{COPY}} = \text{SHA256}\Big(\texttt{"COPY-"} \parallel H_{\text{parent}} \parallel \text{SHA256}(\text{File Contents})\Big) \tag{4}$$

When executing a build instruction, Conductor checks whether a layer directory matching H_{layer} already exists under `.cache/layers/`. If found, the layer is reused immediately without re-executing commands or downloading packages. If an earlier step is modified, its altered hash transitively invalidates all subsequent child layers.

3.3 C Microservice Request Dispatcher

The backend service in Task 4 (`task4/counter-service`) includes a custom in-memory HTTP router written in C without external routing frameworks:

- **DJB2 Hash Algorithm**: Calculates hash values for URI strings using:

$$h_i = (h_{i-1} \ll 5) + h_{i-1} + c = h_{i-1} \times 33 + c \tag{5}$$

- **Linear Probing Open Addressing**: Collisions in the fixed prime-sized hash table ($N = 23$) are resolved via:

$$\text{Index}_k = (\text{Hash}(\text{URI}) + k) \pmod{N} \tag{6}$$

- **Declarative Dispatch Macros**: Handlers are registered at startup using preprocessor macros (`FUNCTION_MAPS`), mapping endpoints such as `/get_counter`, `/get_and_increment_counter`, and `/reset_counter` to function pointers.

4. Software-Defined Container Networking

Conductor provides full container networking without relying on external network plugins or background management daemons:

4.1 Virtual Ethernet Interface Configuration

When `conductor.sh addnetwork` is called, Conductor creates a `veth` pair. One interface (`<name>-outside`) resides in the host root namespace, while the peer interface (`<name>-inside`) is moved into the container's network namespace.

The system assigns deterministic /24 IPv4 subnets from the private 192.168.0.0/16 range, allocating 192.168.k.1 to the host end and 192.168.k.2 to the container end. Before adding routes, Conductor polls the interface status (`wait_for_dev`) until kernel IPv6 Duplicate Address Detection (DAD) completes.

4.2 Routing, NAT, and Peering Policies

- **Outbound WAN Connectivity**: When configured with `-i/--internet`, the host enables IPv4 forwarding and applies an iptables `MASQUERADE` rule on the outbound host interface, along with bidirectional `FORWARD` rules.
- **Port Exposure (DNAT)**: When invoked with `-e/--expose inner-outer`, Conductor creates dual destination NAT rules:

```
# PREROUTING: handles external network traffic
iptables -t nat -A PREROUTING -p tcp -d $HOSTIP \
         --dport $OUTER -j DNAT --to-destination $CONT_IP:$INNER
# OUTPUT: handles host-local traffic
iptables -t nat -A OUTPUT -p tcp -d $HOSTIP \
         --dport $OUTER -m addrtype --src-type LOCAL \
         --dst-type LOCAL -j DNAT --to-destination $CONT_IP:$INNER
```

- **Container Peering**: Inter-container communication is established via `conductor.sh peer A B`, inserting bidirectional `FORWARD` filter rules between the two containers' host-side veth interfaces.

5. Systems Debugging and Implementation Challenges

Developing container tools directly against kernel interfaces exposes several edge cases and subtle implementation hurdles:

5.1 Modern Linux Mount API Compatibility Regression

On modern Linux kernels (Linux 6.8+ with `util-linux 2.42`), `conductor.sh run` failed during OverlayFS initialization with generic mount errors:

```
mount: .../merged: wrong fs type, bad option, bad superblock on overlay
```

Kernel tracing via `strace` revealed that modern `mount(8)` binaries use the new filesystem configuration system calls (`fsopen`, `fsconfig`, `fsmount`). When colon-delimited multi-directory paths were passed to `fsconfig("lowerdir", ...)`, the kernel overlay driver returned `EINVAL`.

To resolve this across environments without modifying the container engine scripts, we enforced the legacy `mount(2)` system-call execution path by exporting:

```
export LIBMOUNT_FORCE_MOUNT2=always
```

5.2 Anonymous NetNS Registration in `iproute2`

When network namespaces are created via `unshare --net`, the kernel creates an anonymous namespace inode accessible under `/proc/[pid]/ns/net`. However, the standard `ip -n <netns>` utility expects named entries in `/var/run/netns/`. We resolved this by linking the namespace file descriptor:

```
mkdir -p /var/run/netns
ln -sf /proc/$UNSHARE_PID/ns/net /var/run/netns/$CONTAINER_NAME
```

This allows standard network configuration utilities to target the container's network stack directly.

5.3 Host-Originated Packet Traversal for Exposed Ports

A common limitation in container port forwarding is that iptables `PREROUTING` rules apply only to incoming external packets and are bypassed by packets generated locally on the host. By introducing a secondary `OUTPUT` chain DNAT rule targeting local socket emissions, local test commands (e.g., `curl hostIP:port`) correctly translate to the container endpoint.

6. Design Trade-offs and Architectural Analysis

- **CLI Scripting vs. Compiled Daemon**: Implementing the engine in Bash allows direct orchestration of standard Linux utilities (`unshare`, `nsenter`, `iproute2`, `iptables`) without maintaining long-running background daemons. While this simplifies inspection and eliminates daemon overhead, it relies on subprocess execution rather than in-memory state tracking.
- **OverlayFS vs. Full Directory Duplication**: Early assignment scaffolding used recursive file copies (`cp -a`) to prepare root filesystems. Migrating to OverlayFS union mounts eliminated rootfs copy latency and enabled shared read-only image layers, at the cost of managing layer metadata and hash chains.
- **Process Discovery via Inspection**: Commands such as `exec` and `stop` identify container processes by scanning the process table (`ps -ef | grep <path>`) rather than maintaining persistent PID files. While effective for simple environments, tracking explicit PID files improves robustness in concurrent multi-tenant workloads.
- **Single-Threaded State Handling**: The backend C counter microservice operates on an unsynchronized in-memory counter. To maintain correctness without introducing mutex overhead or lock contention, the service was scoped to single-threaded execution.

7. Reproduction and Execution Guide

The repository includes automated Makefile targets and manual CLI workflows for running and testing each task:

- **Prerequisites**: Linux system with root access, cgroups v2, OverlayFS, and standard networking utilities (`gcc`, `debootstrap`, `iproute2`, `iptables`, `nsenter`, `unshare`).
- **Task 1 (Namespace Demo)**:

```
make build-task1 && sudo make run-task1
```

- **Task 2 (Chroot and Cgroup Container)**:

```
make build-task2 && sudo make run-task2
```

- **Task 3 (Conductor Image Build and Runtime)**:

```
sudo bash task3/conductor.sh build myimage task3/Conductorfile
sudo bash task3/conductor.sh run myimage mycontainer
sudo bash task3/conductor.sh ps
sudo bash task3/conductor.sh exec mycontainer -- /bin/bash
sudo bash task3/conductor.sh stop mycontainer
```

- **Task 4 (Multi-Container Microservice Deployment)**:

```
sudo make run-task4
# Test from host terminal:
curl http://<hostIP>:3000
```

- **Teardown and Cleanup**:

```
sudo make stop-task4 && sudo make clean
```

8. Conclusion

The Conductor project illustrates the foundational mechanics of Linux containerization. By incrementally composing kernel namespaces, cgroups v2 resource capping, OverlayFS union filesystems, and virtual networking devices, Conductor establishes a functional container engine from first principles. The implementation demystifies container runtimes, highlighting the interaction between operating system abstractions and user-space management tools.